

Multi – Level linked list
–Insert After Element
–Time Complexity Analysis

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}

void insertAfterElement()
{
    int target, data, type;
    cout << "Enter the element to search for: ";
    cin >> target;

    // Simple search (Only searches main level for
    // simplicity in this demo)
    Node *temp = head;
    while (temp != nullptr && temp->data != target)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Element not found in main list." << endl;
        return;
    }
}
```

```

    cout << "Enter data to insert: ";
    cin >> data;
    Node *newNode = createNode(data);

    cout << "Insert as (1) Next Node or (2) Child Node? ";
    cin >> type;

    if (type == 1)
    {
        // Standard Linked List Insertion
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        // Multilevel Insertion
        // If a child already exists, we link the new node
to the front of the child list
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else
    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}
.....

```

```

int main()
{
    int choice;
    while (true)
    {

        cout << "4. Insert After Element (Link or Child)" <<
endl;
        cin >> choice;

        switch (choice)
        {
            case 4:
                insertAfterElement();
                break;
            default:
                cout << "Invalid choice" << endl;
        }
    }
    return 0;
}

```

Time Complexity

1)Best Case when ``Target is found immediately at first node`` $\rightarrow \Omega(1)$.

2)Worst Case ,when ``Element is not found.`` $\rightarrow O(n)$.

Average Case

Inserted position from 1 to $N + 1$, while loop runs from 1 to $N - 1$.

And when the position is uniformly distributed from 1 to $(N + 1)$:

(Average Case i. e. Undetermined Position)

→ If the Element is inserted after 1st node, the loop's inner statement runs 0 time.

→ If the Element is inserted after 2nd node, the loop's inner statement runs 1 time.

→ If the Element is inserted after 3rd node, the loop's inner statement runs 2 time.

→ If the Element is inserted after 4th node, the loop's inner statement runs 3 time.

.

.

→ If the Element is inserted at $(n + 1)^{th}$ node, the loop's inner statement runs $N - 1$ time.

$$\therefore, \sum_{i=0}^{N-1} i = \frac{N(N-1)}{2}$$

Now, If there is 'N' no. of nodes there we can insert data at 2 to $N + 1$ positions or repositioning 1 to N .

For position 1, the probability of insertion is $\frac{1}{N}$.

For position 2, the probability of insertion is $\frac{1}{N}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N}$.

Hence probability of insertion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations)

$$= \frac{1}{N} \times \left(\sum_{i=0}^{N-1} i \right)$$

$$= \frac{1}{N} \times \frac{N(N-1)}{2}$$

$$= \frac{N-1}{2}$$

Applying , Big Θ on the above equation:

$$= \frac{1}{2} \times \Theta(N) - \Theta\left(\frac{1}{2}\right)$$

$$= \Theta(N)$$

Including Insertion Statements

After search completes:

```
newNode->next = temp->next;  
temp->next = newNode;
```

Or,

```
newNode->next = temp->child;  
temp->child = newNode;
```

Takes constant amount of time i. e. $\Theta(1)$.

Therefore , $\Theta(n) + \Theta(1) = \Theta(n)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:

(Simplest approach)

***(If the elements in the linked list are uniformly distributed
and the key is equally likely to be any element in the list)***

***Note: –The checking that Element or Key is found or not , done in the
condition of while loop, and the condition checks `n` times .***

–if key is found at 1st Node , Key/Element searched for 1 time .

–if key is found at 2nd Node , Key/Element searched for 2 time .

–if key is found at 3rd Node, Key/Element searched for 3 time .

.

.

–if key is found at Nth Node , Key/Element searched for N time .

As , the key is equally likely to be any element in the list,

Probablity of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

Probablity of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

Probablity of finding Nth element (p_n), if key equals to Nth element = $\frac{1}{N}$.

Therefore,

$$\text{Probability}(P) = p_1 = p_2 = \dots = p_n = \frac{1}{N}.$$

Therefore, Average Case = Number of times the Key is searched $\times$ probablity of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\therefore \text{Average Case} = \frac{1}{N} \times \left(\sum_{i=1}^N i \right)$$

$$= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right]$$

$$= \frac{N+1}{2}$$

$$= \frac{N}{2} + \frac{1}{2}$$

Applying , Big Theta Θ we get :

$$= \frac{1}{2} \times (\Theta(N) + \Theta(1))$$

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

Multi – level Linked List	Cases	Time Complexity
Insert After Element	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(n)$
	3. Average Case	$\Theta(n)$
